List of Members _____

FINAL PROJECT RUBRIC
OPERATING SYSTEM SIMULATOR

Programming Language: Python or Java | GUI-Based Application | 10-Minute Presentation

I. Project Overview

Develop an interactive Operating System Simulator using Python or Java with a graphical user interface (GUI). The application must implement the required OS concepts and algorithms, accept dynamic user input, display understandable results, and be submitted as source code plus a runnable application.

II. Required Modules and Functionality

Required Module	Expected Functionality
1. CPU Scheduling	Implement all CPU scheduling algorithms covered in class (for example, FCFS, SJF, Priority, and Round Robin). Display scheduling order and relevant calculations such as waiting time and turnaround time.
2. Banker's Algorithm	Allow users to specify process and resource counts. Calculate Need, determine whether the system is safe, and display a safe sequence when one exists.
3. Paging	Simulate page-to-frame mapping and logical-to-physical address translation.
4. Page Replacement	Implement all required page replacement algorithms (for example, FIFO, LRU, and Optimal). Display frame changes, page hits, and page faults.
5. Segmentation	Allow users to define segments, base addresses, and limits. Validate logical addresses and show valid translations or out-of-range errors.
6. Fragmentation	Allow users to enter multiple memory blocks and processes. Demonstrate allocation and identify internal and/or external fragmentation as appropriate.
7. Disk Scheduling	Implement all disk scheduling algorithms covered in class. Display request service order and total head movement.
8. Multiprogramming	Simulate multiple programs or processes sharing memory or system resources, and show relevant system status or utilization.

III. Mandatory Requirement: Dynamic User Input Without Fixed Item Limits

Users must be able to enter a flexible number of data items. The program must not rely on a small hardcoded dataset or impose an arbitrary fixed number of entries. The practical limit is the available memory and processing capacity of the computer.

Module	Dynamic Input Expectation
CPU Scheduling	Users can add a flexible number of processes and enter process IDs, arrival times, burst times, and priorities where applicable.
Banker's Algorithm	Users specify the number of processes and resource types, then enter allocation and maximum-demand data.
Paging / Page Replacement	Users enter a variable-length page reference string and choose the number of frames.
Segmentation	Users define the number of segments and each segment's base address and limit.
Fragmentation	Users enter a variable number of memory blocks and processes.
Disk Scheduling	Users enter a variable-length list of disk requests and the initial head position; disk limits are specified where required.
Multiprogramming	Users define the number of programs or processes and their relevant memory/resource requirements.

Input validation is required. The application should handle blank fields, invalid numbers, negative values where not allowed, and inconsistent input without crashing. Students must test each applicable module with both a small dataset and a substantially larger dataset.

IV. Detailed Grading Rubric (100 Points)

OPERATING SYSTEM RUBRICS

A. Program Development and Implementation — 40 points

Criterion	Excellent	Satisfactory	Needs Improvement	Pts
CPU Scheduling	All required algorithms work correctly and display scheduling results.	Most work; minor errors.	Several missing or incorrect algorithms.	6
Banker's Algorithm	Correct Need calculation, safety check, and safe sequence when one exists.	Basic calculations work; some errors.	Safety checks/calculations mostly incorrect.	5
Paging	Correct page/frame mapping and address translation.	Basic paging with minor errors.	Incomplete or incorrect simulation.	4
Page Replacement	All required algorithms correctly show hits, faults, and frame changes.	Most work; minor errors.	Several algorithms/results incorrect.	6
Segmentation	Correct segment table, base/limit handling, and address validation.	Basic operation with minor errors.	Translation/validation incorrect.	4
Fragmentation	Correct allocation and identification of relevant fragmentation.	Basic allocation; some inaccuracies.	Allocation/results incorrect.	4
Disk Scheduling	All required algorithms show correct service order and head movement.	Most work; minor errors.	Several algorithms/calculations incorrect.	6
Multiprogramming	Demonstrates multiple programs/processes and resource sharing/utilization.	Basic simulation with limited functionality.	Incomplete or unclear simulation.	5

Subtotal: 40 points

B. GUI Design and Usability — 10 points

Criterion	Excellent	Satisfactory	Needs Improvement	Pts
Interface design	Clear, organized, consistent layout.	Generally organized; minor issues.	Confusing or poorly arranged.	2
Navigation and controls	Easy navigation; input fields and controls work.	Most controls work.	Controls confusing or often fail.	2
Output presentation	Readable tables, charts, diagrams, or results.	Results displayed but need organization.	Unclear or incomplete results.	1
Error handling	Validates inputs and gives useful messages.	Handles common errors only.	Invalid input often causes failure.	2
Dynamic input flexibility	Flexible number of entries across applicable modules; no arbitrary fixed item limit.	Variable input supported but with restrictions or validation issues.	Hardcoded data or fixed sizes prevent flexible simulation.	3

Subtotal: 10 points

C. Algorithm Accuracy and Testing — 15 points

Criterion	Excellent	Satisfactory	Needs Improvement	Pts
Algorithm correctness	All algorithms follow correct procedures.	Minor logical errors.	Major logical errors.	6
Test cases	Tests normal, boundary, and relevant exceptional cases; small and larger datasets.	Basic tests; limited coverage.	Insufficient testing.	4
Result verification	Checks results against manual/known expected results.	Some results verified.	Results not adequately verified.	3
Stability	Runs consistently without unexpected crashes.	Occasional minor issues.	Frequent crashes or failures.	2

Subtotal: 15 points

D. Source Code and Program Submission — 10 points

Criterion	Excellent	Satisfactory	Needs Improvement	Pts
Source code	Complete, readable, organized, appropriately commented.	Understandable with minor issues.	Disorganized or difficult to understand.	3
Runnable application	Runnable program plus required files/dependencies and instructions.	Runs with minor setup issues.	Cannot run reliably or essential files missing.	3
Program organization	Appropriate functions,	Basic organization.	Difficult to maintain.	2

	classes, or modules.			
Documentation	Instructions, algorithm descriptions, and sample inputs/outputs.	Incomplete but usable.	Little or no useful documentation.	2

Subtotal: 10 points

E. Project Presentation and Demonstration — 15 points

Criterion	Excellent	Satisfactory	Needs Improvement	Pts
Project overview	Clearly explains purpose, scope, and features.	General overview with omissions.	Cannot explain project clearly.	2
Live demonstration	Demonstrates GUI and required modules successfully.	Demonstrates most modules.	Incomplete/unsuccessful demo.	5
Algorithm explanation	Explains algorithms and how results are obtained.	Basic explanation; details missing.	Limited understanding shown.	3
Time management	Covers key requirements within 10 minutes.	Slightly over/under time with omissions.	Poor time management.	2
Communication	Clear presentation and accurate answers.	Adequate; struggles with some questions.	Difficulty explaining or answering.	3

Subtotal: 15 points

F. Understanding of Operating System Concepts — 10 points

Criterion	Excellent	Satisfactory	Needs Improvement	Pts
Conceptual understanding	Correctly explains purpose and behavior of OS concepts.	Mostly understands; minor misconceptions.	Limited understanding.	4
Justification of results	Explains outputs and compares algorithms.	Basic explanations.	Cannot justify results adequately.	3
Questions and answers	Independently answers technical questions and explains decisions.	Answers basic questions with prompting.	Cannot explain major implementation parts.	3

Subtotal: 10 points

V. Final Score Summary

Assessment Component	Points
A. Program Development and Implementation	40
B. GUI Design and Usability (includes dynamic input)	10
C. Algorithm Accuracy and Testing	15
D. Source Code and Program Submission	10
E. Presentation and Demonstration	15
F. Understanding of OS Concepts	10
TOTAL	100

VI. Suggested 10-Minute Presentation Schedule

Time	Activity
1 minute	Introduction: project title, objectives, language, and purpose.
2 minutes	GUI and structure: main menu, navigation, inputs, and outputs.
4 minutes	Live demonstration: show selected modules and briefly cover all eight areas.
2 minutes	Algorithm explanation: calculations, result comparison, and one challenge.
1 minute	Questions and conclusion.

VII. Submission Checklist

☐ Source code (Python .py files or Java source files).

☐ Runnable application/package (for example, a Java JAR or packaged Python executable if required).

☐ Required libraries/dependencies and setup/run instructions.

☐ Documentation describing each module and its algorithms.

☐ Sample inputs and outputs or screenshots.

☐ Test evidence for a small dataset and a substantially larger dataset for each applicable module.

☐ GUI supports flexible user-defined input without an arbitrary fixed item limit.

☐ Program validates input and handles errors without crashing.

VIII. Instructor Evaluation Sheet

Student/Group Name: _____

Assessment Component	Maximum Score	Score Awarded

Program Development and Implementation	40	_____
GUI Design and Usability	10	_____
Algorithm Accuracy and Testing	15	_____
Source Code and Program Submission	10	_____
Presentation and Demonstration	15	_____
Understanding of OS Concepts	10	_____
TOTAL	100	_____

Instructor's Remarks:

_____

Date: _____

Instructor Reminder

Require students to submit the source code and runnable application before the final demonstration. During the presentation, at least one technical question about an algorithm and explain its input, processing, and output. A GUI alone is not sufficient: the program must perform the actual calculations and simulations.